Case Study

Deep Learning Models for Water Stage Predictions in South Florida

Jimeng Shi[1]; Zeda Yin, S.M.ASCE[2]; Rukmangadh Sai Myana[3]; Khandker S. Ishtiaq[4]; Anupama John, M.ASCE[5]; Jayantha Obeysekera, M.ASCE[6]; Arturo S. Leon[7]; and Giri Narasimhan[8]

Abstract: Simulating and predicting the water level/stage in river systems is essential for flood warnings, hydraulic operations, and flood mitigations. Physics-based detailed hydrological and hydraulic computational tools, such as the Hydrologic Engineering Center's River Analysis System (HEC-RAS), MIKE, and the storm water management model, can be used to simulate a complete watershed and compute the water stage at any point in the river system. However, these physics-based models are computationally intensive, especially for large watersheds and for longer simulations, since they use detailed grid representations of terrain elevation maps of the entire watershed and solve complex partial differential equations for each grid cell. To overcome this problem, we train several deep learning (DL) models for use as surrogate models to rapidly predict the water stage. A portion of the Miami River in South Florida was chosen as a case study for this paper. Extensive experiments show that the performance of various DL models (MLP, RNN, CNN, LSTM, and RCNN) is significantly better than that of the physics-based model, HEC-RAS, even during extreme precipitation conditions (i.e., tropical storms), and with speedups exceeding 500×. To predict the water stages more accurately, our DL models use both measured variables of the river system from the recent past and covariates for which predictions are typically available for the near future. **DOI: 10.1061/JWRMD5.WRENG-6194.**

Practical Applications: Simulating and predicting the water level/stage in river systems is essential for flood warnings, hydraulic operations, and flood mitigations. However, the current physics-based computational tools are based on solving complex partial differential equations and are time consuming, especially for large watersheds and for longer simulations. To overcome this problem, we aimed to train deep learning (DL) surrogate models to rapidly predict the water stages. Experiments show that trained DL models (MLP, RNN, CNN, LSTM, and RCNN) achieved a speedup of at least 500x in comparison to the physics-based models while achieving comparable or better error rates. Furthermore, the DL models performed better than standard machine learning models and standard linear regression methods. We also investigate these models in our work under both normal and extreme conditions (i.e., tropical storms). The use of future covariates is another contribution, which resulted in better accuracy for these models.

Author keywords: Deep learning; Physics-based model; Water level/stage prediction; Future covariates.

Introduction

Floods seasonal in many areas but are mostly rare natural events caused by increased flow rate and water stage in a river, resulting in temporary submerging of low-lying areas along its banks (Parhi 2018). Floods result in commercial losses and dangers to human life (Grothmann and Reusswig 2006; Hirabayashi and Kanae 2009; Alexander et al. 2019) and could contribute to environmental and public health risks (Okaka and Odhiambo 2018; Rivett et al. 2022). Thus, knowing the time and location of floods ahead of time is critical for hydraulic structure operators to make informed decisions for flood mitigation promptly (Kusler 2004; Leon et al. 2020) and for citizens and local governments to be better prepared for potential flood outcomes. Higher extremes in rainfall due to climate change will likely increase the frequency and volume of floods (Seneviratne 2022). Currently, physics-based models are available to simulate river floods, including the Hydrologic Engineering Center's River Analysis System (HEC-RAS), MIKE developed by the Danish

[1]Ph.D. Student, Knight Foundation School of Computing and Information Sciences, Florida International Univ., 11200 SW 8th St., Miami, FL 33199. Email: jshi008@fiu.edu

[2]Ph.D. Candidate, Dept. of Civil and Environmental Engineering, Florida International Univ., 10555 W Flagler St., Miami, FL 33174. Email: zyin005@fiu.edu

[3]Ph.D. Student, Knight Foundation School of Computing and Information Sciences, Florida International Univ., 11200 SW 8th St., Miami, FL 33199. ORCID: https://orcid.org/0009-0003-8748-4238. Email: rmyan001@fiu.edu

Note. This manuscript was submitted on March 23, 2023; approved on March 25, 2025; published online on July 18, 2025. Discussion period open until December 18, 2025; separate discussions must be submitted for individual papers. This paper is part of the ***Journal of Water Resources Planning and Management***, © ASCE, ISSN 0733-9496.

[4]Hydrologist, Everglades Foundation, Palmetto Bay, FL 33157; formerly, Postdoctoral Scholar, Sea Level Solutions Center, Florida International Univ., 11200 SW 8th St., Miami, FL 33199. ORCID: https://orcid.org/0000-0003-2130-2908. Email: kishtiaq@fiu.edu

[5]Postdoctoral Scholar, Institute of Environment, Florida International Univ., 11200 SW 8th St., Miami, FL 33199. Email: anujohn@fiu.edu

[6]Professor, Sea Level Solutions Center, and Dept. of Earth and Environment, Florida International Univ., 11200 SW 8th St., Miami, FL 33199. ORCID: https://orcid.org/0000-0002-7038-1668. Email: jobeysek@fiu.edu

[7]Associate Professor, Dept. of Civil and Environmental Engineering, Florida International Univ., 10555 W Flagler St., Miami, FL 33174. Email: arleon@fiu.edu

[8]Ryder Endowed Professor, Knight Foundation School of Computing and Information Sciences, Florida International Univ., 11200 SW 8th St., Miami, FL 33199 (corresponding author). Email: giri@fiu.edu

Hydraulic Institute company, and the storm water management model (SWMM). However, thousands of simulations of these physics-based models may be required to find the optimal flow release on the control of hydraulic structures (e.g., gates, pumps, dams, and reservoirs) to mitigate the floods (Leon et al. 2020, 2021). In real-time applications, the use of conventional physics-based models is infeasible since they are extremely time-consuming by simulating each cross section in the river systems, especially for large complex domains with multiple driving factors, e.g., heavy rain, strong winds, and storm surges. Therefore, there is a critical need for a surrogate model that can rapidly and accurately predict the water stages at specific locations along the river.

Several studies have explored the application of various machine learning (ML) and deep learning (DL) models for a range of purposes (Yin et al. 2022). The artificial neural network (ANN) has been applied to forecast river flow in the Apalachicola River (Huang et al. 2004) and net inflow volumes into Lake Okeechobee (Obeysekera et al. 2000). Huang et al. 2004 compared the results of the forecasted flow rates from the ANN model with those from a traditional autoregressive integrated moving average forecasting model. Sadler et al. (2018) used a random forest model to predict flood severity in Norfolk, VA. Given extensive environmental data (i.e., rainfall, tide, groundwater table level, and wind conditions) as input, they focused on a classification task by predicting the number of flood reports. Furthermore, Sapitang et al. (2020) utilized four ML models to enhance daily forecasts of reservoir water levels and reported the ML intramodel comparison. The ML models used in their work include boosted decision tree regression, decision forest regression, Bayesian linear regression, and neural network regression. Similarly, Ayus et al. (2023) employed random forest, XGBoost, and bidirectional long short-term memory (LSTM) network models on 30 years of data to predict daily water levels in Jezioro Kosno Lake in Poland.

As shown above, ML and DL approaches have been widely applied to model environmental data. However, we noted the following limitations. First, most efforts directly tried off-the-shelf ML models, which may be limiting their ability to model increasingly complex datasets. Second, most only reported errors in comparison with observed data or simply provided ML intramodel comparison (Sapitang et al. 2020), but did not report the comparison with the performance of current physics-based models. Third, some of them (Sapitang et al. 2020; Ayus et al. 2023) worked on coarse time granularity, e.g., daily, weekly, or even monthly. More importantly, none of these methods incorporated future covariates into their models, especially the covariates that can be estimated or determined in advance, e.g., rainfall information and control settings of hydraulic structures.

In this work, we aim to address the limitations above. Our main contributions are as follows.

- We show that DL models perform better than physics-based models such as HEC-RAS for prediction tasks. While the methods are comparable in prediction accuracy, DL methods are orders of magnitude faster than their physics-based counterparts, opening the possibility for real-time flood management. They also outperform simpler regression and standard ML methods.
- To fine-tune our understanding of the applicability of DL methods to this type of data, we applied multiple DL models (MLP, RNN, CNN, and LSTM) to predict the water stages. We also proposed a combinational model, RCNN, by combining recurrent neural networks (RNNs) and convolutional neural networks (CNNs) in series. We concluded that the differences in performance between the DL models were not statistically significant.
- We conducted extensive experiments to study the impact of varying forecast lead times. We proved that RCNN marginally outperforms other DL models over forecast lead times.
- We demonstrated that incorporating future covariates into DL models can lead to improved predictions, even if they are low in accuracy. To simulate low accuracy forecasts, we also performed our experiments after adding random noise to the future covariates.

Methodology and Modeling

In this section, we present the approach used. We introduce data acquisition, data processing, the HEC-RAS models, and the DL models. Both single and combinational DL models are explored in this paper. HEC-RAS is a widely used simulation software to model the hydraulics of water flow through natural rivers and other channels and is well-suited to our study domain—the coastal watershed in South Florida, for which we obtained a HEC-RAS model. Thus, HEC-RAS became our representative physics-based model to be used as a benchmark for comparisons.

Study Area and Data Set

We tested our methodologies with a sample dataset from the South Florida water management district.

Study Area

South Florida is vulnerable to hurricane events with extremely heavy precipitation (Azzi et al. 2020). The Miami River is one of the largest rivers in South Florida emptying into the Biscayne Bay. The city of Miami with a sizable population and many commercial enterprises is located along the river. A schematic of the South Florida region is shown in Fig. 1. The study area consists of the last 5.6 mi of the Miami River, its two tributaries—C4 (upper) and C6 (lower), three water stations—S25A, S25B, and S26, and hydraulic structures (i.e., spillway gates, pumps) used to manage the flows. Pumps can allow additional discharge capacity during high tide or storm surge events. Water station S26 is a water station with a gated spillway and a set of pumps. Station S25B has two spillway gates that work independently and a set of pumps. The gate and pump flow measurements are the total flow through the gate and pump, respectively. Station S25A has a culvert structure (with a control gate), which is recorded. Stations S1 and S4 are water stations used to record and monitor the water stage. The mouth of the river is located on the Biscayne Bay, which demonstrates that the downstream water stage is strongly influenced by the tides in Biscayne Bay. The hydraulic structures avoid seawater flowing back from the Biscayne Bay and thus have a significant impact on the river system's state.

Data Acquisition and Description

The river data used in this paper were collected hourly over a period of 11 years (January 1, 2010, to December 31, 2020) at five water stations (S1, S4, S25A, S25B, and S26 shown in Fig. 1) and retrieved from DBHYDRO, the South Florida Water Management District's (SFWMD's) database. The rainfall data are pixel-based (grided) radar precipitation data retrieved from the Next Generation Weather Radar (NEXRAD) system data archived by SFWMD. The precipitation data were collected every hour using a grid of sensors and averaged over all the grid cells in the domain. Other features, the flows at upstream locations (S25A, S25B, and S26), and the water stage at downstream (S4) were additional covariates that can influence the prediction of the target variables (i.e., water stage at S1 and tail water stage at S25A, S25B, and S26). The data features and usage are described in Table 1, and all these inputs are used for both HEC-RAS and DL models.

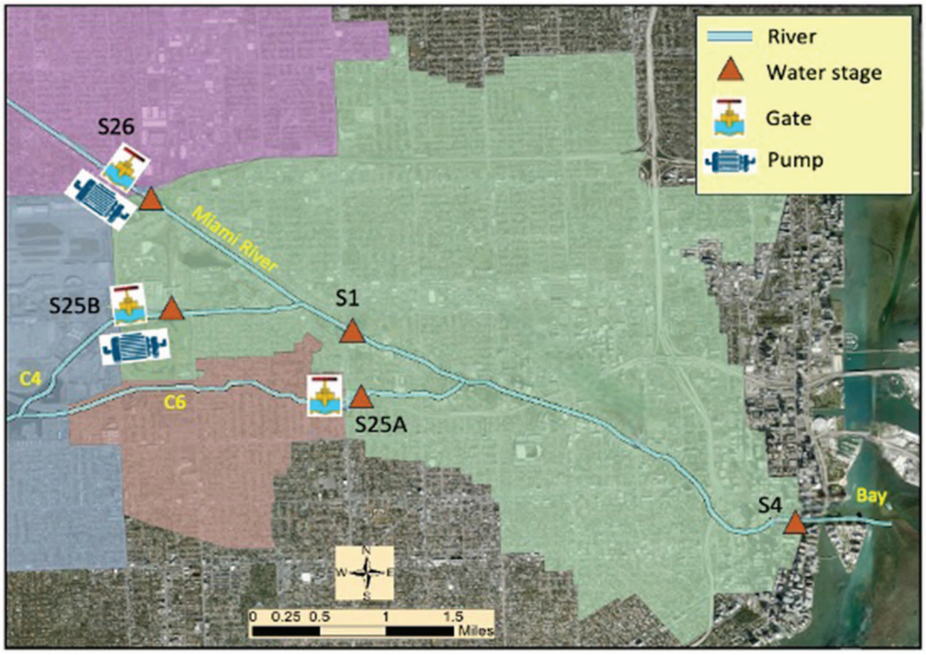

Fig. 1. (Color) Map of the study area, showing the shape of the downstream of Miami River, location of the hydrological stations, and water stage measuring stations. The legend represents the elevation of the Datum NAVD 88. (Map generated using PCSWMM modeling software and SFWMD Geospatial Open Data.)

Table 1. Description of data feature and usage

Description	Abbreviation	Unit	Target variables
S26 gate flow	Flow_S26	cubic feet/second	
S26 pump flow	Pump_S26	cubic feet/second	
S26 tailwater stage	TWS_S26	cubic feet/second	Target
S25A gate flow	Flow_S25A	cubic feet/second	
S25A tailwater stage	TWS_25A	feet	Target
S25B total gate flow	Flow_S25B	cubic feet/second	
S25B pump flow	Pump_S25B	cubic feet/second	
S25B tailwater stage	TWS_S25B	feet	Target
S1 water stage	WS_S1	feet	Target
S4 water stage (tide info)	WS_S4	feet	
Mean gridded precipitation intensity	Grid_Rainfall	inches/hour	

Data Preprocessing for Deep Learning Models

Unlike for the HEC-RAS model, data for the DL models need to be preprocessed beforehand. The data were normalized to avoid gradient values from exploding or vanishing (i.e., becoming infinity or zero). The min–max normalization technique (Patro and Sahu 2015) was used as shown in Eq. (1)

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{1}$$

where the minimum value was set to 0, the maximum was set to 1, and all other values were linearly scaled as shown in Eq. (1). The data set was divided into 80% training (January 1, 2010, to August 7, 2018) and 20% test set (August 8, 2018, to December 31, 2020).

HEC-RAS Model and Other Baseline Models

HEC-RAS was used to simulate a benchmark physics-based model. In this work, a 1D–2D coupled HEC-RAS model was used to calculate the overland flow more accurately. Four boundary conditions were required to run the HEC-RAS model: three upstream conditions (at S26, S25A, and S25B) and one downstream condition (at S4). All three upstream conditions were set as flow hydrographs. The upstream flow hydrograph was set to the gate flow value, with an added pump flow value if there was a pump station. In HEC-RAS, the average of the gridded precipitation data over all grids in the domain was added to the boundary conditions at Station S26. We have the water stage at S4 as the downstream boundary condition for HEC-RAS. Since the HEC-RAS model is a numerical solver for shallow water equations, the input data (boundary conditions) of HEC-RAS had the same period ($t + 1$ to $t + k$) as the output data, and there was no lag time data needed in the model. A 2-year-long simulation (from January 1, 2019, 00:00, to December 31, 2020, 23:00) was completed using HEC-RAS and was used to compare with the prediction results of DL models. The computation timestep size was set at 1 min to keep the computation stable, and the output interval was set the same as for the DL computations (i.e., 1 h). The HEC-RAS modeling framework is provided in Fig. S1.

We implemented the basic linear regression (LR) model to use as a baseline statistical approach. We also chose XGBoost as a baseline ML model. XGBoost was among the best performing ML models in prior experiments with this dataset (data not shown).

Deep Learning Models

Four DL model architectures were used in this paper, including RNNs, LSTM networks, multilayer perceptron (MLP), and CNNs. A DL model combining RNN and CNN in series was also developed to achieve better performance. The detailed architecture and mathematical representation of each model are outlined in Figs. S2–S6. The overall modeling framework for the DL models is shown in Fig. 2. The significant difference between the DL models and HEC-RAS is that the DL methods depend on past information (e.g., looking back w timesteps of all the variables), future

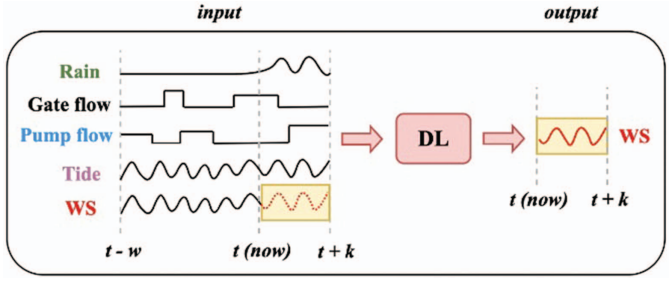

Fig. 2. (Color) Overall framework of the DL models. Vertical dashed lines indicate time points of interest. Input data are shown as a set of time series to the left of the schematic (except water stage from time $t+1$ to $t+k$), while the output to predict is shaded yellow.

predictable information (e.g., k timesteps of rainfall data in the future), and human-controlled information (e.g., flow rate scheduled through the gate or pump for the k timesteps), while HEC-RAS only requires the last two. In other words, HEC-RAS only needs the boundary conditions for the prediction period, while DL models were designed to use both the (future) information from the desired period as well as the data from the recent past.

In Fig. 2, we assume that the current time is represented by time t, and w and k are the lengths of the history considered and the size of forecast lead time, respectively. WS represents the water stages at S1, S25A tailwater, S25B tailwater, and S26 tailwater. Gate flow refers to the amount of water flowing through the gate, and pump flow represents the amount of pumped water, both representing a time series. Tide is the water stage at S4; i.e., the sea stage information. For each location, we seek to predict the water stage from time $t+1$ to $t+k$. Therefore, all covariates (rain, tide, gate flow, and pump flow) from $t+1$ to $t+k$ were considered using predicted or scheduled information. Rain and tide forecasts are available from the weather forecasting agency and oceanographic agency, while gate flow and pump flow are assumed to have been provided by the water management agency. DL models are also provided with historical data (rain, gate flow, pump flow, and water stage) from time $t-w$ to t, with the assumption that recent past information informs us of the *state of the system*. Mathematically, the modeling process is

$$\mathrm{WS}^{i}_{[t+1,t+k]} = f(\mathrm{WS}_{[t-w,t]}, \mathrm{Tide}_{[t-w,t+k]}, \mathrm{Rain}_{[t-w,t+k]}, \mathrm{GF}_{[t-w,t+k]}, \mathrm{PF}_{[t-w,t+k]}) \quad (2)$$

where $\mathrm{WS}^{i}_{[t+1,t+k]}$ = water stage at the ith station from time $t+1$ to $t+k$, $i \in \{S1, S25A, S25B, S26\}$; $\mathrm{WS}_{[t-w,t]}$ = vector representing water stages for all four locations from time $t-w$ to t; $\mathrm{Tide}_{[t-w,t+k]}$ = water stage at S4 from time $t-w$ to $t+k$; $\mathrm{Rain}_{[t-w,t+k]}$ = mean gridded rainfall for the study domain from time $t-w$ to $t+k$; and $\mathrm{GF}_{[t-w,t+k]}$ and $\mathrm{PF}_{[t-w,t+k]}$ = water flow through gate and pump, respectively, for all four stations from time $t-w$ to $t+k$. We set the length of past window [also called look-back window (Nie et al. 2023), rolling window (Li et al. 2014), and sliding window (Shi et al. 2023)] $w = 72$ h and the size of forecast lead time $k = 24$ h in our work.

Multilayer Perceptron

MLP is part of a family of fully connected feedforward neural networks (Naskath et al. 2023). MLP usually consists of at least three layers: one input layer, one or more hidden layers, and one output layer. The input layer is used to load input features to be processed. Hidden layers and the output layer are used for intermediate nonlinear computations and processing the predicted output, respectively. By stacking multiple layers of neurons, where each layer applies a nonlinear activation function, the MLP can learn complex, nonlinear mappings from inputs to outputs (Del Campo et al. 2021). The composition of these nonlinear functions through the network allows it to approximate highly intricate functions. However, since MLPs are fully connected neural networks, they are prone to overfit the training data, leading to poor generalization on new, unseen data (Zelený and Fryza 2023). Therefore, *regularization* and *dropout* techniques are generally used to mitigate overfitting by simplifying the model structure.

Recurrent Neural Networks

Unlike traditional feedforward neural networks, RNNs have connections that form directed cycles, allowing them to maintain a *memory* of previous inputs (Rezk et al. 2020). This is particularly useful for tasks where the context from previous inputs is important. They are well suited to recognize patterns in sequences of data, such as time series, text, speech, or video. The temporal dependencies can be learned by recurrently training and updating the transitions of an internal (hidden) state from the last timestep to the current timestep. Nevertheless, RNNs have the following limitations. Since RNNs learn knowledge iteratively over timesteps, training RNNs can be challenging due to the vanishing or exploding gradient problem, especially for long sequences (Kag and Saligrama 2021). Specifically, the gradients at the last timesteps are difficult to back-propagate to the timesteps at the beginning. This phenomenon is also called "memory forgetting" (Ghojogh and Ghodsi 2023), caused by the inherent characteristic of RNNs—sequential modeling.

Long Short-Term Memory

RNNs are susceptive to forget the knowledge from the past due to vanishing gradients, LSTM networks were proposed as a variant of RNNs (Shi et al. 2022; Rithani et al. 2023) to mitigate this problem. At time point t, an LSTM includes a cell state, $\boldsymbol{C}_t$, which runs straight along the entire chain with only some linear interactions. LSTM has hidden states, $\boldsymbol{S}_t$, representing the information from the past timesteps and the information that is filtered/updated with the new input x_t by the following gate operations. Generally, LSTM can learn slightly longer time dependencies by incorporating forget gates, input gates, additional gates, and output gates to control the inflow, filtering, and outflow of past and current information. The above gates can help the models to alleviate the vanishing gradient problem to some extent. However, LSTMs are more complex than traditional RNNs due to the multiple gates and state variables, leading to longer training times (Yu et al. 2019).

Convolutional Neural Networks

CNNs, traditionally used for image and video processing, have also been proven effective for time series forecasting. When applied to time series data, CNNs can capture temporal patterns and dependencies by treating the time series as a one-dimensional sequence. Convolutional layers apply filters (kernels) that slide over the input sequence to detect patterns (Borovykh et al. 2018). Each filter is a small, learnable array that extracts features from the time series. Each convolutional layer is usually followed by a maximum pooling layer, which is used to extract the most meaningful values of each convolutional computation. When multiple time series (features) are provided, the input can be represented as a two-dimensional array (timesteps × features). This allows the CNN models to capture and learn the relationships across different variables (Keren and Schuller 2016).

In summary, CNNs excel at capturing local patterns in data since each filter has a local receptive field, meaning it looks at a small portion of the input at a time. More interestingly, CNNs apply

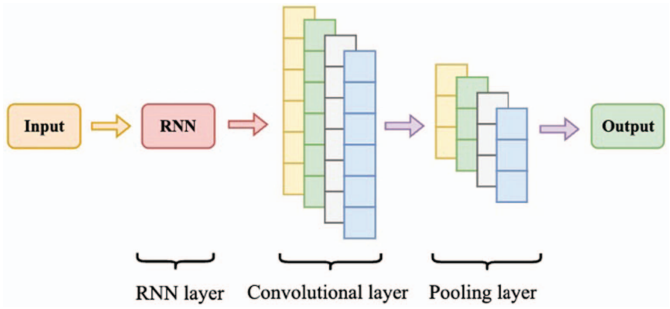

Fig. 3. (Color) Architecture of the recurrent convolutional neural networks.

filters uniformly across the input, so their ability to detect patterns is not affected by the length of the sequence (Alzubaidi et al. 2021). They do not rely on propagating information through a hidden state, so the prediction quality remains consistent regardless of sequence length. On the other hand, CNNs may not capture sequential dependencies as effectively as RNNs due to the limited size of local receptive field. More details of CNNs can be found in Keren and Schuller (2016) and Borovykh et al. (2017).

Combination of RNN and CNN: RCNN

To leverage the strengths of both RNNs and CNNs while mitigating their weaknesses, an approach is to use an RNN first, followed by a CNN (see Fig. 3). This configuration can be particularly advantageous for time series forecasting and other sequential tasks. Initially, an RNN can process the sequential data, capturing temporal patterns and sequential dependencies. The RNN's hidden states at each timestep encapsulate the temporal dynamics of the sequence. These hidden states, which now contain rich temporal information, can then be treated as a new sequence of features. Next, a CNN can be applied to this sequence of RNN-generated features. The convolutional layers in the CNN can then detect local patterns and spatial hierarchies within the sequence of RNN features. This combination allows the CNN to extract meaningful patterns from the temporally aware features produced by the RNN, enhancing the model's ability to recognize both short-term and long-term dependencies.

Results

For DL models, we used all the measured variables from the past 168 timesteps (7 days) and all the predictable and determinate covariates (precipitation and gate releases) from the future 24 timesteps (1 day) as input data, with the intent to predict target variables (water stage at S1, S25A, S25B, and S26) for the future 24 timesteps (1 day). After training DL models with the training set, we applied these models to the test set (2019–2020) and compared the predicted water stages with the simulated water stages of HEC-RAS and the observed water stages from the measuring stations. In the following discussion, we represent the kth predicted timestep in the future with $t + k$ hour prediction. For example, $t + 24$ h prediction refers to the predicted timestep 24 h in the future.

Performance Analysis

Performance Analysis on Prediction Timesteps

DL models were set to predict the target variables from $t + 1$ h to $t + k$ hours. In this subsection, we present the prediction results for the forecast lead times $t + 1$, $t + 12$, $t + 18$, and $t + 24$ h. The experimental results are reported using *perfect foresight* future covariates (tide and rainfall). We have included LR and a machine learning model (XGBoost) as baselines. While the physics-based model, HEC-RAS, has also been included, we note that it does not provide "predictions" in the true sense of the term with arbitrary forecast lead times. Instead, it provides simulations using "perfect foresight" covariates at that future time. Consequently, the error in simulation for HEC-RAS is independent of the lead time. In contrast, all machine learning models can, in theory, predict with arbitrary lead times with or without the availability of future covariate estimations. Metrics including the mean absolute error (MAE), root mean square error (RMSE), Nash–Sutcliffe model efficiency coefficient (NSE), and Kling–Gupta efficiency (KGE) were used to measure the difference between observed and predicted data. NSE and KGE are goodness-of-fit indicators widely used in hydrologic sciences for comparison between simulations to observations, while MAE and RMSE are widely used in the machine learning community

$$\text{MAE} = \frac{\sum_{i=1}^{N} |y_i - \hat{y}_i|}{N} \tag{3}$$

$$\text{RMSE} = \sqrt{\frac{\sum_{i=1}^{N} (y_i - \hat{y}_i)^2}{N}} \tag{4}$$

$$\text{NSE} = 1 - \frac{\sum_{i=1}^{N} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{N} (y_i - \bar{y}_i)^2} \tag{5}$$

$$\text{KGE} = 1 - \sqrt{(r-1)^2 + (\alpha - 1)^2 + (\beta - 1)^2} \tag{6}$$

where y_i, $\hat{y}_i$ = observed data and model outputs; $\bar{y}_i$ = mean of observed water levels; and N = number of samples. In Eq. (6), r = Pearson correlation coefficient; α = term representing the variability of prediction errors; and β = bias term.

Table 2 shows the performance as a function of forecast lead time. The initial observation reveals that DL models outperform linear regression and traditional machine learning models. This underscores their superior capability to learn and capture complex nonlinear relationships within the data. Among DL models, RNN and LSTM models were more sensitive to the forecast lead time, performing well for short-term predictions (1 and 8 h), but suffered larger errors for longer-time predictions. In contrast, MLP and CNN were relatively insensitive to forecast lead time, displaying stable predictions with lower variance even for longer predictions. The combined model (RCNN) produced the least MAE and in four out of five cases for the RMSE measure. We surmise that RCNNs can simultaneously learn the cross-time and cross-variable relationships from data.

In our experiments above, we used the actual future observed values for the future covariates, but they are estimated with some uncertainties in real scenarios. To mimic the uncertainty in future estimates, researchers often add random Gaussian noise to them as an uncertainty proxy (Tabas and Samadi 2022; Iliopoulou and Koutsoyiannis 2020). Therefore, we also performed experiments after adding 20% random noise to these covariates over the entire future k timesteps during the test phase. The magnitude of the noise signal utilized in our experiments was suggested by the work on quantitative precipitation forecast by SFWMD (Seo et al. 2012). We observed that the fluctuation of prediction errors by using the DL methods (see Table 3) is relatively small (<4%). Even when 40% random noise values were added as a proxy to the forecast estimates, the errors roughly doubled (data not shown). However, the error percentages remained low yet, suggesting that the DL models capture the underlying relationships in the data

Table 2. Performance comparison of DL models at all locations (S1, 25A, S25B, and S26) for different forecast lead times

Metrics	Models	1-h ahead	8-h ahead	16-h ahead	24-h ahead	Entire 24 h
MAE (ft)	HEC-RAS	0.186	0.186	0.186	0.186	0.186
	LR	0.275	0.199	0.261	0.288	0.296
	XGBoost	0.184	0.187	0.188	0.192	0.187
	MLP	0.161	0.167	0.148	0.157	0.151
	RNN	**0.075**	**0.093**	0.145	0.148	0.120
	LSTM	0.098	0.147	0.165	0.152	0.148
	CNN	0.102	0.120	0.118	0.124	0.118
	RCNN	0.097	0.094	**0.099**	**0.106**	**0.098**
RMSE (ft)	HEC-RAS	0.238	0.238	0.238	0.238	0.238
	LR	0.256	0.318	0.317	0.347	0.332
	XGBoost	0.270	0.273	0.274	0.284	0.274
	MLP	0.206	0.205	0.183	0.197	0.191
	RNN	**0.098**	0.126	0.185	0.192	0.158
	LSTM	0.124	0.198	0.218	0.202	0.200
	CNN	0.130	0.153	0.151	0.157	0.150
	RCNN	0.118	**0.122**	**0.130**	**0.141**	**0.129**
NSE	HEC-RAS	0.932	0.932	0.932	0.932	0.932
	LR	0.920	0.877	0.878	0.854	0.866
	XGBoost	0.912	0.910	0.910	0.903	0.910
	MLP	0.948	0.949	0.959	0.953	0.956
	RNN	**0.988**	0.980	0.958	0.955	0.969
	LSTM	0.981	0.953	0.942	0.951	0.951
	CNN	0.979	0.971	0.972	0.970	0.972
	RCNN	0.980	**0.982**	**0.979**	**0.976**	**0.980**
KGE	HEC-RAS	**0.950**	**0.950**	**0.950**	**0.950**	**0.950**
	LR	0.735	0.841	0.770	0.710	0.731
	XGBoost	0.830	0.822	0.820	0.816	0.821
	MLP	0.882	0.880	0.893	0.888	0.887
	RNN	0.958	0.951	0.890	0.907	0.920
	LSTM	0.922	0.895	0.885	0.910	0.895
	CNN	0.924	0.898	0.913	0.907	0.905
	RCNN	0.933	0.936	0.931	0.936	0.936

Note: The lowest prediction errors (MAE and RMSE) and highest efficiencies (NSE and KGE) are in bold.

Table 3. MAE changes of the DL models of all locations (S1, 25A, S25B, and S26) for different forecast lead times after adding random noise to the observed precipitation

Models	1-h ahead (%)	8-h ahead (%)	16-h ahead (%)	24-h ahead (%)	Entire 24 h (%)
MLP	+0.047	+0.097	+0.027	−0.005	+0.050
RNN	+0.001	−0.004	−0.015	−0.014	−0.001
LSTM	−0.005	+0.000	+0.001	+0.003	−0.002
CNN	+1.222	+0.535	+3.320	+2.139	+1.615
RCNN	+0.116	+0.118	+0.128	+0.061	+0.105

rather than relying on specific, possibly noisy, patterns in the training data.

To analyze the distribution of prediction errors, violin plots were used to visualize the prediction errors for 24 h in the future (see Fig. 4). Violin plots display the density distribution along with the median value, the peak value, the 25th percentile, and the 75th percentile (see the top and bottom of the small black box inside). The larger the highest density of the violin plot, the greater are the errors distributed in that corresponding area. We also observe that the majority of the prediction errors for the RCNN model are centered around 0 ft, predominantly within the narrow range of −0.3 to 0.3 ft. Compared to other DL models, the RCNN model exhibited lower frequency and magnitude for extreme errors. This trend of lower mean and variance in prediction errors remains consistent even as the forecast lead times increase, demonstrating the RCNN model's superior accuracy and stability over time. The superior performance of RCNN was also confirmed by the p values for the nonparametric Wilcoxon test used to compare RCNN with the other four DL models. All comparisons between RCNN and other models were statistically significant with p values <0.01 (see Tables S1 and S2).

Additionally, to analyze the extreme prediction errors quantitively, we reported those extreme errors that are beyond the range (−0.5, 0.5) ft in Table 4. We note that 99% of the absolute prediction errors of the DL models are within 0.5 ft. (Note that our programs allow the threshold to be changed flexibly.) It shows the RCNN model has the lowest percentage (0.803%) of error extremes. More interestingly, RNN and LSTM models generate consistently higher magnitude of error extremes with the increase in forecast lead time while MLP, CNN, and RCNN perform relatively stable.

Comparing the Performance of Models at Different Locations

We present MAE results to study the model performance across different locations. In Table 5, we observed that the DL models display smaller MAE values than HEC-RAS for all locations. The RCNN model outperforms all other models for three of four locations (S1, S25A, and S25B). On the other hand, location S26 has the highest errors for all DL models compared to other locations.

Fig. 5 shows that the DL models perform with lower magnitude of error extremes at location S1 (results for locations S25A, S25B, and S26 are in Fig. S7). Based on the distribution of prediction errors, we observed that the medians of DL models are comparable with those of HEC-RAS. Furthermore, most prediction errors of DL models are in the (−0.25, 0.25) ft range when compared to HEC-RAS (−0.4, 0.4) ft, suggesting a lower variance for the DL models. In Table 6, we computed the percentage of extreme errors beyond the range (−0.5, 0.5) ft.

Result Visualization

In this section, we present visualizations of the observed and predicted water stages at Station S1, which is centrally located near Miami International Airport. The visualization for other stations has been provided in Figs. S11 and S12. We chose two short periods: one without a storm event (April 1, 2019) and another with a storm event (August 1–3, 2020).

Normal Period without a Storm Event

Fig. 6 shows the observed water stages, predicted water stages (using the DL models), and the simulated water stages (using HEC-RAS) at location S1 on April 1, 2019, which had no storm events. The corresponding prediction errors have been also presented below. Both DL models and HEC-RAS models showed good agreement with the observed data. Moreover, DL models show the lower prediction errors, indicating their powerful capacity in water stage prediction.

Period with an Extreme Storm Event

Heavy storm events have great impacts on water stages in the river. Accurately predicting the water stages is critical during such periods. In this work, we used around 9 years of historical data to train the DL models. The training data set included extreme events such as Hurricane Irma (2017), tropical storm Andrea (2013), Hurricane Sandy (2012), and tropical storm Isaac (2012). Tropical storm Isaias (2020)

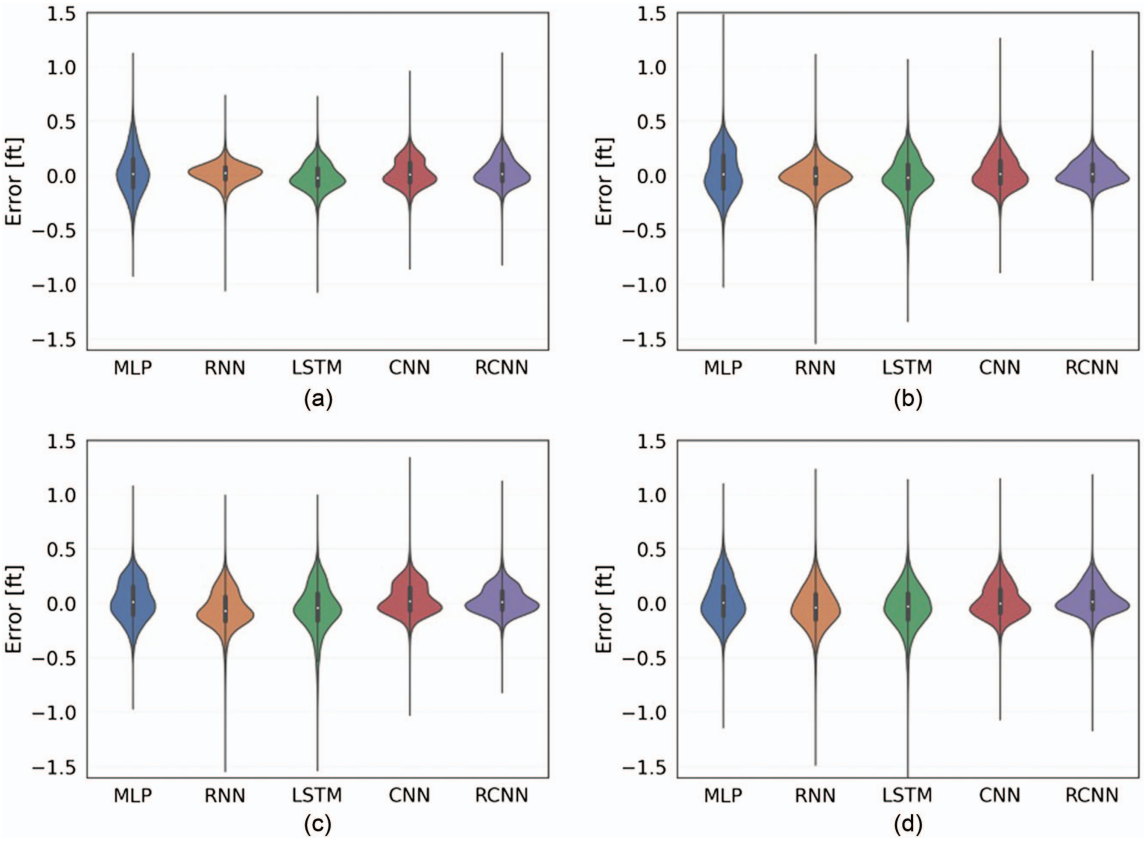

Fig. 4. (Color) Violin plots showing density distributions of actual errors for the five DL methods at all locations with forecast lead time: (a) 1 h; (b) 12 h; (c) 18 h; and (d) 24 h. We zoomed in by magnifying the *y* axis of the violin plots for clear comparison (see Appendix C2.1). Note that the prediction errors in violin plots are reported based on the perfect foresight of future covariates (tide and rainfall).

Table 4. Percentage of prediction errors beyond the range (−0.5, 0.5) ft

Models	1-h ahead (%)	8-h ahead (%)	16-h ahead (%)	24-h ahead (%)	Entire 24 h (%)
MLP	1.684	0.880	0.358	0.990	0.753
RNN	**0.073**	0.464	1.316	1.481	0.777
LSTM	0.110	2.499	3.125	2.237	2.360
CNN	0.086	0.237	0.297	0.291	**0.244**
RCNN	0.127	**0.194**	**0.250**	**0.283**	**0.224**

Note: The lowest percentages are in bold.

Table 5. Mean absolute error (MAE) of the predicted results for 24 h at four different locations for the test set

Models	S1	S25A	S25B	S26
MLP	0.160	0.106	0.105	0.258
RNN	0.135	0.139	0.150	**0.170**
LSTM	0.146	0.136	0.160	0.167
CNN	0.107	0.093	0.090	0.206
RCNN	**0.072**	**0.085**	**0.082**	0.182
HEC-RAS	0.176	0.182	0.198	0.184

Note: The lowest MAE values are in bold.

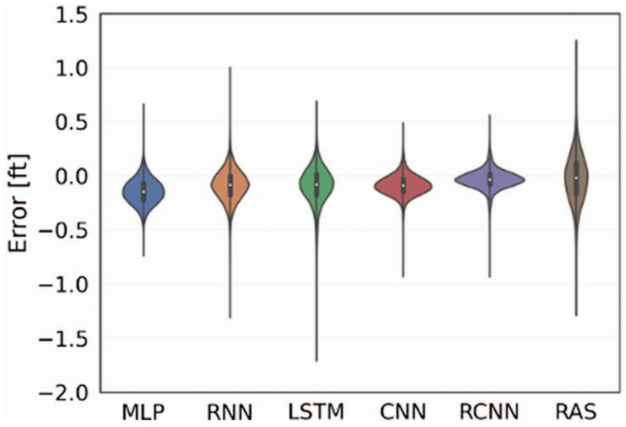

Fig. 5. (Color) Violin plots of error distributions for 24-h-ahead forecasts at locations, S1. All comparisons with HEC-RAS (RAS) were statistically significant at the 0.01 level. Figures in the Appendix C2.2 show differences among these same violin plots but enlarged in the middle to see the differences more clearly.

was used to test the accuracy of the models. Tropical storm Isaias reached the greater Miami region nearby around August 1–3, 2020. Thus, we chose this period for the analysis as well.

The predictions of 24-h ahead during tropical storm Isaias and the corresponding prediction errors are shown in Fig. 7. Compared to the normal conditions, the water stages during tropical storm Isaias are higher. Meanwhile, the MAE values in this set of experiments are slightly higher compared to the experiments without a storm event. However, all DL models still outperformed the HEC-RAS, with RCNN continuing to rank the first among the DL models. Visualizations for locations S25A, S25B, and S26 are provided in Fig. S12.

Table 6. Percentage of prediction errors beyond the range (−0.5, 0.5) ft at four different locations

Models	S1 (%)	S25A (%)	S25B (%)	S26 (%)
MLP	0.208	0.229	0.322	2.847
RNN	0.961	1.455	1.704	1.273
LSTM	2.364	1.320	2.883	1.579
CNN	0.109	**0.208**	**0.218**	0.525
RCNN	**0.094**	0.338	0.327	**0.374**
HEC-RAS	2.599	3.249	4.417	3.585

Note: The lowest percentage values are in bold.

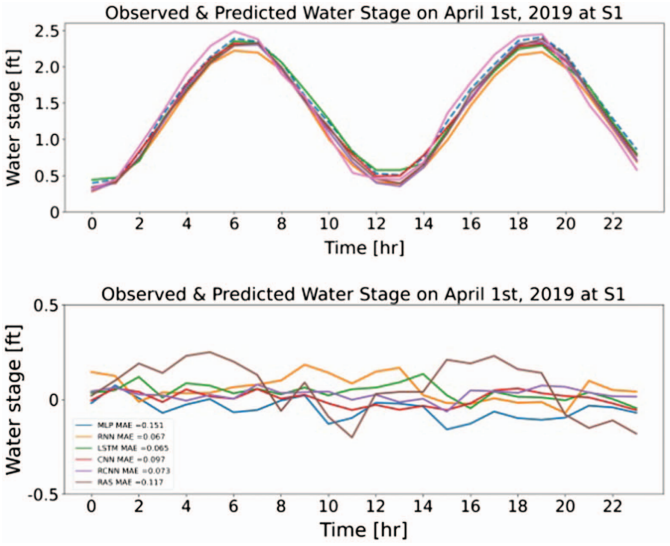

Fig. 6. (Color) Observed and predicted water stage at Station S1 on April 1, 2019.

Table 7. Training and test time for all models

Task	HEC-RAS	MLP	RNN	LSTM	CNN	RCNN
Training time	—	27 min	267 min	1,033 min	167 min	667 min
Test time	45 min	0.34 s	1.81 s	4.81 s	0.62 s	2.41 s

Computation Time

Rapidly predicting the water stage is crucial if these tools are to be used during real-time operations and with different hypothetical boundary conditions (i.e., different gate flow, pump flow, rainfall intensity, and tide). Table 7 compares the run times for the DL models in their test phase with HEC-RAS. The training phase is a one-time cost and needs to be included here.

Discussion and Conclusions

Several DL models were developed and tested to predict water stages at specific locations in the river. Table 7 shows that the execution time of the DL models is much lower than the HEC-RAS model, thus bringing us closer to building real-time prediction tools for more complex tasks. In terms of the model accuracy, all DL models in our paper show good agreement with the observed data, and their performance is better than that of HEC-RAS. In addition, inaccurate predictions can be more dangerous, especially when water levels are high. Overestimation of water stages can cause false alarms and panic, while underestimation of water stages can result in unexpected flooding events. We set the 99th percentile of water stage (i.e., 0.5 ft) as a cutoff and analyzed the prediction errors of underestimation and overestimation. In future work, we aim to deal with the small proportion of error extremes since floods could be occurring even during a short time.

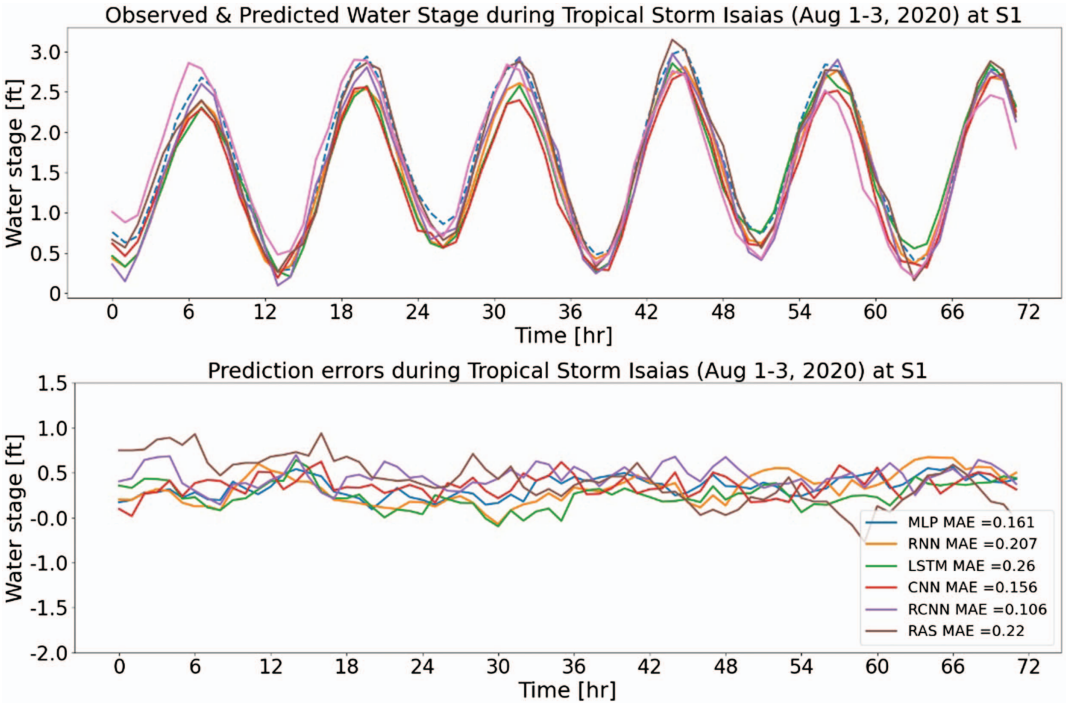

Fig. 7. (Color) Observed and predicted water stages at Station S1 during tropical storm Isaias (2020).

The key conclusions have been listed as follows. Our DL surrogate models (MLP, RNN, CNN, LSTM, and RCNN) can predict water stages at locations of interest with higher accuracy and efficiency than the physics-based model, HEC-RAS. DL models show at least 500x speedup over HEC-RAS. More interestingly, we found that DL models such as RNN and LSTM were more sensitive to the forecast lead time, with higher prediction errors for greater forecast lead time while MLP, CNN, and RCNN models had relatively more stable performance. The combinational RCNN model provides relatively better performance for both short- and long-term prediction. Finally, the utilization of future covariates is essential for the predictions, especially under extreme storm events. Our DL models indicate good robustness even when 20% random noise values were added to future covariates used in this work.

Reproducible Results

Data and code with detailed instructions (readme file) are available in the GitHub repository (Shi 2023), and Zenodo (Shi 2025). An Associate Editor for Reproducibility downloaded the materials from Zenodo, installed the computing environment, and ran the notebooks in the postprocessing folder. He was able to reproduce results in Figs. 4(a–c) to 7. He was able to reproduce all results in Table 4 except for the entry in Row "RCNN" Column "24-h ahead." He was able to reproduce all results in Table 5 except on the final row labeled "HEC-RAS." Rahuul Rangaraj, a Ph.D. student at Florida International University, successfully downloaded all the materials, installed the Python environment, ran the models, and reproduced all figures and tables.

Data Availability Statement

Some or all data, models, or code generated or used during the study are available in a repository or online in accordance with funder data retention policies.

Acknowledgments

This work was partly supported by the National Science Foundation grants numbered 2203292 (to AL) and 2118329 (to GN via University of Illinois Urbana-Champaign).

Author Contributions

Jimeng Shi: Conceptualization; Formal analysis; Methodology; Visualization; Writing – original draft; Writing – review and editing. Zeda Yin: Conceptualization; Data curation; Formal analysis; Methodology; Visualization; Writing – original draft. Rukmangadh Sai Myana: Conceptualization; Methodology. Khandker S. Ishtiaq: Data curation; Methodology. Anupama John: Data curation; Methodology. Jayantha Obeysekera: Conceptualization; Supervision. Arturo S. Leon: Conceptualization; Funding acquisition; Methodology; Supervision; Validation. Giri Narasimhan: Conceptualization; Formal analysis; Funding acquisition; Methodology; Supervision; Validation; Writing – review and editing.

Supplemental Materials

Figs. S1–S12 and Tables S1 and S2 are available online in the ASCE Library (www.ascelibrary.org).

References

Alexander, K., S. Hettiarachchi, Y. Ou, and A. Sharma. 2019. "Can integrated green spaces and storage facilities absorb the increased risk of flooding due to climate change in developed urban environments?" *J. Hydrol.* 579 (Dec): 124201. https://doi.org/10.1016/j.jhydrol.2019.124201.

Alzubaidi, L., J. Zhang, A. J. Humaidi, A. Al-Dujaili, Y. Duan, O. Al-Shamma, J. Santamaría, M. A. Fadhel, M. AI-Amidie, and L. Farhan. 2021. "Review of deep learning: Concepts, CNN architectures, challenges, applications, future directions." *J. Big Data* 8 (Dec): 1–74. https://doi.org/10.1186/s40537-021-00444-8.

Ayus, I., N. Natarajan, and D. Gupta. 2023. "Prediction of water level using machine learning and deep learning techniques." *Iran. J. Sci. Technol. Trans. Civ. Eng.* 47 (4): 2437–2447. https://doi.org/10.1007/s40996-023-01053-6.

Azzi, Z., M. Matus, A. Elawady, I. Zisis, P. Irwin, and A. Gan Chowdhury. 2020. "Aeroelastic testing of span-wire traffic signal systems." *Front. Built Environ.* 6 (Jul): 111. https://doi.org/10.3389/fbuil.2020.00111.

Borovykh, A., S. Bohte, and C. W. Oosterlee. 2017. "Conditional time series forecasting with convolutional neural networks." Preprint, submitted March 14, 2017. https://arxiv.org/abs/1703.04691.

Del Campo, F. A., M. C. G. Neri, O. O. V. Villegas, V. G. C. Sánchez, H. D. J. O. Domínguez, and V. G. Jiménez. 2021. "Auto-adaptive multilayer perceptron for univariate time series classification." *Expert Syst. Appl.* 181 (Nov): 115147. https://doi.org/10.1016/j.eswa.2021.115147.

Ghojogh, B., and A. Ghodsi. 2023. "Recurrent neural networks and long short-term memory networks: Tutorial and survey." Preprint, submitted April 22, 2023. https://arxiv.org/abs/2304.11461.

Grothmann, T., and F. Reusswig. 2006. "People at risk of flooding: Why some residents take precautionary action while others do not." *Nat. Hazard.* 38 (1): 101–120. https://doi.org/10.1007/s11069-005-8604-6.

Hirabayashi, Y., and S. Kanae. 2009. "First estimate of the future global population at risk of flooding." *Hydrol. Res. Lett.* 3: 6–9. https://doi.org/10.3178/hrl.3.6.

Huang, W., B. Xu, and A. Chan-Hilton. 2004. "Forecasting flows in Apalachicola River using neural networks." *Hydrol. Processes* 18 (13): 2545–2564. https://doi.org/10.1002/hyp.1492.

Iliopoulou, T., and D. Koutsoyiannis. 2020. "Projecting the future of rainfall extremes: Better classic than trendy." *J. Hydrol.* 588 (Sep): 125005. https://doi.org/10.1016/j.jhydrol.2020.125005.

Kag, A., and V. Saligrama. 2021. "Training recurrent neural networks via forward propagation through time." In *Proc., Int. Conf. on Machine Learning*, 5189–5200. New York: Proceedings of Machine Learning Research.

Keren, G., and B. Schuller. 2016. "Convolutional RNN: An enhanced model for extracting features from sequential data." In *Proc., 2016 Int. Joint Conf. on Neural Networks (IJCNN)*, 3412–3419. New York: IEEE.

Kusler, J. 2004. *Multi-objective wetland restoration in watershed contexts*. Technical Rep. Portland, ME: Association of State Wetland Managers.

Leon, A. S., L. Bian, and Y. Tang. 2021. "Comparison of the genetic algorithm and pattern search methods for forecasting optimal flow releases in a multi-storage system for flood control." *Environ. Modell. Software* 145 (Nov): 105198. https://doi.org/10.1016/j.envsoft.2021.105198.

Leon, A. S., Y. Tang, L. Qin, and D. Chen. 2020. "A MATLAB framework for forecasting optimal flow releases in a multi-storage system for flood control." *Environ. Modell. Software* 125 (Mar): 104618. https://doi.org/10.1016/j.envsoft.2019.104618.

Li, L., F. Noorian, D. J. Moss, and P. H. Leong. 2014. "Rolling window time series prediction using MapReduce." In *Proc., 2014 IEEE 15th Int. Conf. on Information Reuse and Integration*, 757–764. New York: IEEE.

Naskath, J., G. Sivakamasundari, and A. A. S. Begum. 2023. "A study on different deep learning algorithms used in deep neural nets: MLP SOM and DBN." *Wireless Pers. Commun.* 128 (4): 2913–2936. https://doi.org/10.1007/s11277-022-10079-4.

Obeysekera, J., P. Trimble, L. Cadavid, P. R. Santee, and C. White. 2000. "Use of climate outlook for water management in South Florida, USA."

In *Proc., Engineering Jubilee Conf*. West Palm Beach, FL: South Florida Water Management District.

Okaka, F. O., and B. Odhiambo. 2018. "Relationship between flooding and out break of infectious diseasesin Kenya: A review of the literature." *J. Environ. Public Health* 2018 (1): 5452938. https://doi.org/10.1155/2018/5452938.

Parhi, P. K. 2018. "Flood management in Mahanadi Basin using HEC-RAS and Gumbel's extreme value distribution." *J. Inst. Eng. India Ser. A* 99 (4): 751–755. https://doi.org/10.1007/s40030-018-0317-4.

Patro, S. G. K., and K. K. Sahu. 2015. "Normalization: A preprocessing stage." Preprint, submitted March 19, 2015. https://arxiv.org/abs/1503.06462.

Rezk, N. M., M. Purnaprajna, T. Nordström, and Z. Ul-Abdin. 2020. "Recurrent neural networks: An embedded computing perspective." *IEEE Access* 8 (Mar): 57967–57996. https://doi.org/10.1109/ACCESS.2020.2982416.

Rithani, M., R. P. Kumar, and S. Doss. 2023. "A review on big data based on deep neural network approaches." *Artif. Intell. Rev.* 56 (12): 14765–14801. https://doi.org/10.1007/s10462-023-10512-5.

Rivett, M. O., et al. 2022. "Acute health risks to community hand-pumped groundwater supplies following Cyclone Idai flooding." *Sci. Total Environ.* 806 (Feb): 150598. https://doi.org/10.1016/j.scitotenv.2021.150598.

Sadler, J. M., J. L. Goodall, M. M. Morsy, and K. Spencer. 2018. "Modeling urban coastal flood severity from crowd-sourced flood reports using Poisson regression and random forest." *J. Hydrol.* 559 (Apr): 43–55. https://doi.org/10.1016/j.jhydrol.2018.01.044.

Sapitang, M., M. Ridwan, W. Faizal Kushiar, K. Najah Ahmed, and A. El-Shafie. 2020. "Machine learning application in reservoir water level forecasting for sustainable hydropower generation strategy." *Sustainability* 12 (15): 6121. https://doi.org/10.3390/su12156121.

Seneviratne, S. 2022. "Weather and climate extreme events in a changing climate." In *IPCC sixth assessment report*. Cambridge, UK: Cambridge University Press.

Seo, D. J., R. Siddique, G. Shaughnessy, S. Huebner, S. Noorjahan, and J. Raymond. 2012. "Probablistic rainfall forecasting using single-valued rainfall forecasts for risk-based water management at the south Florida water management district." In *Proc., 10th Int. Conf. on Hydroscience and Engineering*, edited by S. Hagen, M. Chopra, K. Madani, S. Medeiros, and D. Wang. Karlsruhe, Germany: The HENRY Hydraulic Engineering Repository.

Shi, J. 2023. "GitHub repository." Accessed June 30, 2023. https://github.com/JimengShi/DL-WaLeF.

Shi, J. 2025. "Zenodo repository." Accessed January 18, 2025. https://zenodo.org/records/14715390.

Shi, J., M. Jain, and G. Narasimhan. 2022. "Time series forecasting using various deep learning models." *Int. J. Comput. Syst. Eng.* 16 (6): 224–232. https://doi.org/10.48550/arXiv.2204.11115.

Shi, J., R. Myana, V. Stebliankin, A. Shirali, and G. Narasimhan. 2023. "Explainable parallel RCNN with novel feature representation for time series forecasting." Preprint, submitted December 20, 2023. https://arxiv.org/abs/2305.04876.

Tabas, S. S., and S. Samadi. 2022. "Variational Bayesian dropout with a Gaussian prior for recurrent neural networks application in rainfall–runoff modeling." *Environ. Res. Lett.* 17 (6): 065012. https://doi.org/10.1088/1748-9326/ac7247.

Yin, Z., L. Zahedi, A. S. Leon, M. H. Amini, and L. Bian. 2022. "A machine learning framework for overflow prediction in combined sewer systems." In *Proc., World Environmental and Water Resources Congress*, 194–205. New York: Proceedings of Machine Learning Research.

Yu, Y., X. Si, C. Hu, and J. Zhang. 2019. "A review of recurrent neural networks: LSTM cells and network architectures." *Neural Comput.* 31 (7): 1235–1270. https://doi.org/10.1162/neco_a_01199.

Zelený, O., and T. Fryza. 2023. "Multi-branch multi layer perceptron: A solution for precise regression using machine learning." In *Proc., 2023 33rd Int. Conf. Radioelektronika*, 1–5. New York: IEEE.